

What a Semantic Observer Needs from the Agent Runtime

An interface statement for runtime and platform engineers. It says where the observer stands, what it needs to see, what it needs to be able to do, what it emits, and what it refuses to claim.

1. Purpose and scope

This document states what a meaning-slip observer needs from an agent runtime in order to operate beside it, so that a runtime team can decide whether to expose it. It discloses nothing beyond the published framework paper, *Meaning Slip: A Framework for AI Conversation Governance* (Mossrake Group, July 2026, rev. August 2026), which describes the instrument's architecture in substance: a proxy between agents and their model providers, a deposition taken from each party to an exchange, a comparison of the two accounts, and an append-only record of what was found.

The interface is the subject here. The observer's internal design (how exchanges are linked without identifiers, how comparison is calibrated, how the reader is qualified) is a separate specification available under NDA, named in the last section and not described.

The document is written for runtime and platform engineers, with NVIDIA OpenShell as the worked case because its supervisor middleware contract (RFC 0009, tracked in [issue #1733](#), which superseded the content-inspection proposal in [#1272](#)) defines the exact seam: registered services that inspect supervisor-managed data paths at named hooks. Nothing here is specific to OpenShell; any runtime that proxies model calls and stamps trace identifiers can expose the same interface.

2. The gap this fills

A secure runtime governs what an agent can reach: files, processes, credentials, network destinations, and at L7 the method, path and operation of a call. It does not govern what the agent understood from what it read, and by its own account it does not inspect request or response bodies for semantic signals ([#1272](#) opens by saying so). That is the correct division. Enforcement should not depend on a model's reading of a message.

What is left ungoverned is the exchange: the event in which text produced by one agent's model call becomes part of the input to another agent's model call. Two agents can read the same exchange differently and proceed, each executing its own reading, while every enforcement check passes. The paper calls the accumulating displacement between the two readings *meaning slip*. It is not a property of any message, so a per-message check cannot see it, and not a property of any endpoint, so endpoint monitoring cannot either. It is a property of the pair, and it lives in what each party separately holds.

The observer is the instrument for that layer. It sits above enforcement, never in its path, and produces a record of what each party understood, at the moment it understood it, from a copy. It makes no inference about intent, and the runtime's policy decisions are inputs to its record, not outputs of it. The two layers are complementary by construction: the runtime bounds what the agent can do, the observer records what the agent took the exchange to mean, and neither claims the other's coverage.

3. Where the observer stands

The observer is a consumer of copies, not a hop in the call. The agent's call reaches the provider through the runtime's proxy exactly as it does today; the observer receives a copy of the request and response after the fact and talks to the provider on its own credential when it needs to ask a party a question.

The observer sits beside the proxy, never in the call path

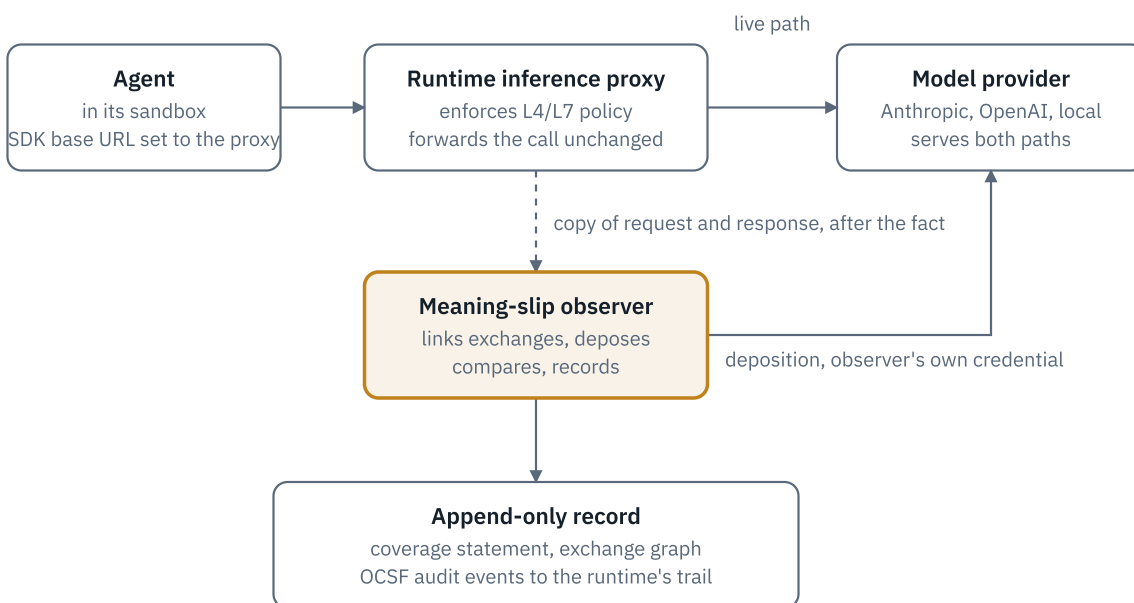


Figure 1. Observer placement: one live path, two side paths. The live path is the runtime's; the observer cannot delay, block or alter a call.

Two placements deliver the same copies. As a **sidecar to the runtime's proxy**, the observer receives each request and response pair through a hook the runtime exposes; this is the natural fit where the runtime already buffers bodies. As a **proxy in its own right**, agents' base URLs point at the observer, which forwards to the runtime's proxy or directly to the provider; this fits deployments whose runtime exposes no hook. In both placements the observer is an HTTP proxy or a consumer of one, and a request the proxy cannot carry (per-host signed traffic, such as AWS SigV4) is delivered by an in-process adapter instead, or named on the coverage statement as unseen.

4. What it needs to see

The observer needs the complete request body and the complete response of every model call it is to cover, as bytes, unaltered. The request body is the whole statement of what a party was shown at that moment; nothing else about the party is observable, and a redacted or summarised copy is not a copy.

NEED	WHY	WITHOUT IT
Full request body (system prompt, messages, tool declarations)	The party's context is its state; a deposition replays it	No deposition; observation only
Full response, including tool-use blocks	The writer's output is what crosses to the reader	Exchanges cannot be linked from content
Streamed responses reassembled, or delivered chunk by chunk with order preserved	Most agent traffic streams	Partial outputs; broken links
The served model version from the response, not only the requested model	Replay must address the version that answered	Depositions against the wrong model
Provider and wire dialect (Anthropic Messages, OpenAI chat completions, OpenAI Responses)	Fields differ by dialect	Misparsed context
Whether the call was wire-complete or store-anchored (a <code>previous_response_id</code> pointer)	Decides replay versus in-place deposition	Wrong deposition method, or a false claim of coverage
Timestamps at receipt of request and at completion of response	Exchanges are ordered by the reader's call	Ambiguous ordering
Response bodies in a readable encoding, or delivered compressed with the encoding named	Provider SDKs request gzip by default	Most inference responses invisible

Store-anchored calls, where the provider holds the conversation and the request carries a pointer, are the one case where the wire is not the whole context. The observer needs the pointer and the new turn, and needs to know that the history was not on the wire, so that it can depose in place rather than replay, or record the deposition as unavailable. A runtime that reports this per call, rather than leaving the observer to infer it, closes the most common gap in coverage.

5. What it needs to know

The observer's primary way of knowing which call belongs to which run, and which call caused which, is to read identifiers the deployment's own tracing already stamps on its calls. It does not require a deployment to add one, and where identifiers are absent it infers from content, structure and timing, with every inference carrying its basis and a confidence class. Identifiers turn an inference into a fact, and the coverage statement reports how much of the picture rests on each.

IDENTIFIER	WHAT IT ESTABLISHES	WHERE A RUNTIME ALREADY HAS IT
Trace or run id, shared across the calls of one task	Which calls belong together	Distributed tracing headers on outbound calls
Parent span or causal id, naming the call that led to this one	Writer to reader, exactly, whatever the wording	Orchestration frameworks that nest spans
Sandbox or agent identity, stable across a session	Which lineage a call belongs to; the party type	The runtime's own sandbox id and policy scope
Hand-off or artifact id, when code passes a message between agents	Links a hop the observer cannot see, because it is not a model call	Message buses, task queues, shared stores
Policy decision reference on the call (allowed, denied, what scope)	Correlates a reading with what enforcement did about it	The runtime's audit trail

The last row is the one only a secure runtime can supply, and it is the one that makes the two layers more than adjacent. A reading the observer records beside the policy decision the runtime made on the same call gives an incident reviewer both halves of the story: what the agent understood, and what it was permitted.

Where identifiers are absent, the observer's inference is deliberately conservative: it prefers to leave an exchange unlinked rather than claim a link its evidence does not support, and on terse, templated or fields-only traffic it will leave most of them unlinked. That is by design: a link that did not happen, once on the record, poisons everything after it. The practical consequence for a runtime is that one identifier on each call is worth more than any amount of inference, and the coverage statement will say exactly how much more.

6. What it needs to be able to do

The observer needs an outbound path to each model provider, on its own credential, so that it can take an isolated deposition: send a party's copied context back to the model that served it, with the party's own response appended and one question added, and record the answer. The party's live conversation is not touched and the party is not told. The observer's calls therefore appear on the deployment's provider dashboards under the observer's key, are rate-limited on that key, and can be turned off per provider without affecting any agent.

Three things follow for the runtime.

1. **The observer's egress is a policy object.** Its calls to providers should be scoped exactly like an agent's: named destinations, named credentials, nothing else. A runtime that already expresses network scope as policy can express the observer's scope the same way, and the observer's coverage statement will name any provider it was not permitted to reach.
2. **Store-anchored deposition needs same-scope credentials.** Where a provider holds the conversation, the observer asks the provider to continue the stored conversation with a question added and an instruction not to save the result. Providers refuse this across scope boundaries (for OpenAI, across projects). The runtime should either issue the observer a credential inside the agents' scope, or accept that those depositions are recorded as unavailable with the provider's reason.
3. **No credential, no deposition, and the record says so.** A deployment that gives the observer copies but no keys still gets linking, the exchange graph and transmission

checks. Depositions are recorded as not offered, per provider. The observer never substitutes a weaker question for a deposition it cannot take.

The observer also needs to know, per agent, whether the agent's understanding is formed in its model context at all. An agent whose reasoning is spread across code, local state and narrow model calls that were never given the conversation has no copy that can be questioned; it is observed at its message interface instead, and deposited there only if an operator opts it in, because a message to a live agent is a message it may act on. That decision is the operator's; the observer reports the signals and does not make it.

7. Non-interference guarantees

The observer gives the runtime the following guarantees, and a runtime can hold it to them, because each is checkable from the runtime's side of the interface.

- It never delays, blocks, modifies or replays an agent's call on the live path. Copies are consumed asynchronously; if the observer is slow or down, the call proceeds and the copy is dropped and counted.
- It never sends a message to an agent, except a deposition through an agent's message interface that an operator has set up in advance for that agent.
- It never injects text into a party's context. Depositions are taken from copies, on the observer's own credential, and are marked so that a provider does not store them as part of the party's conversation.
- It never lets a model decide whether meaning slipped. A model may read a text and extract fields from it; the finding is a deterministic comparison against a calibrated band, and the model's version is on every call that contributed.
- It never claims coverage it did not have. Every unlinked exchange, unavailable deposition, refused credential and unseen hop is counted on the coverage statement.

The first guarantee is the one that lets the observer coexist with an enforcement layer whose whole value is being on the path. Enforcement is synchronous and must be; observation is asynchronous and must be. A runtime should be able to verify, from its own

metrics, that the observer's presence adds no latency to any call and that removing it changes nothing an agent can see.

8. What it emits

The observer produces four things, and the runtime's audit trail can consume the first of them today.

OUTPUT	CONTENTS	HOW A RUNTIME CONSUMES IT
Audit events	One event per exchange linked, deposition taken, comparison made and slip found, each with its basis and confidence class. Never a body: bodies are referenced by SHA-256 hash and live only in the observer's own record	Emitted as OCSF events into the runtime's existing audit trail, so slips sit beside policy decisions in one timeline
Coverage statement	What was seen, what was inferred, what was deposited, what could not be seen, per provider and per pair type	Read by operators; the honest denominator for every other number
Exchange graph	Which agents talk to which, how often, on which model versions, and how much of what crosses creates obligations	Diffed against the deployment's declared architecture; produced from the first hour of traffic with no inference or calibration
Append-only record	Every request copy, exchange, deposition and comparison, hash-chained	Retained under the deployment's own policy; the material an incident review or a third-party-risk team reads

The record is the artifact that model governance cannot produce after the fact. A transcript says what was said; the readings, what each party understood, were never in it, and the contexts that held them are discarded as the system runs. The record is made while the parties exist. It is also the record that makes a sustained misrepresentation expensive over time: every deposition is a committed statement, and a party whose statements must stay consistent with all its past statements and all its later conduct is a party for whom the lie gets harder with every check.

What the record does not contain is a verdict. It says two accounts diverged, by how much, and how the divergence surfaced. Whether that was a blameless fork on the task or something about the counterparty itself is a reading made from the record afterward, by a person, and the record is kept so that reading can be made and revised without the record inheriting it.

9. Proposed observe phase

OpenShell's data-plane extensibility is supervisor middleware (RFC 0009): a registered service implementing `openshell.middleware.v1.SupervisorMiddleware`, advertising bindings of operation and phase, and returning typed findings. The current bindings are `HTTP_REQUEST / PRE_CREDENTIALS`, which delivers the buffered request body after policy admits the call and before credential injection, and `HTTP_RESPONSE / PRE_RETURN`, which delivers the response head at preflight and then the body, whole or in ordered units of at most 64 KiB, before the sandbox receives it. A `request_id` correlates the two evaluations. Nearly everything in sections 4 and 5 is therefore already on the wire.

OpenShell's own guidance for observation is to allow the traffic and report findings. An observer can be built that way today, and the interface statement would be dishonest if it did not say so. What that construction costs is specific, and it is the whole case for a separate phase.

COST OF OBSERVING AS A PASS-THROUGH MIDDLEWARE

SOURCE

Every response unit is evaluated in lockstep: the relay awaits the stage's result before forwarding the next unit. Streaming inference flushes small units, so the observer adds one round trip per flush on the live path

`response.rs` ,
`process_units_from`

`on_error` applies. `fail_closed` makes an observer outage a sandbox outage; `fail_open` emits a bypass finding at High severity, so an observer restart reads as a security event

RFC 0009, failure policy and observability sections

A request body over the 4 MiB platform cap follows `on_error` ; growing inference contexts are the RFC's own example

RFC 0009, risks

Body inspection is unavailable for compressed responses, and provider SDKs request compression by default. A request-stage removal of `Accept-Encoding` works around it at the cost of bandwidth

Supervisor middleware docs, operations

Per-stage timeouts (500 ms default) and a 30 s chain budget are sized for gates, not for a consumer that may want to persist bytes

RFC 0009, protocol section

The phase

`SUPERVISOR_MIDDLEWARE_PHASE_OBSERVE` , valid for `HTTP_REQUEST` and `HTTP_RESPONSE` . It reuses the existing messages; what changes is when they are sent and what a result may do.

PROPERTY	PROPOSED	WHY
Delivery	After the relay has forwarded the unit (request: after upstream write; response: after downstream write). The relay never awaits an observe stage	Observation must be asynchronous; enforcement must be synchronous
Results	Findings and metadata only. Body transforms, header mutations, deny and block are invalid for the phase and are dropped and counted, not failed	An observer that could block is a gate with extra latency
Failure	No <code>on_error</code> . An unavailable or slow observe stage is a coverage state, recorded like <code>binding_not_selected</code> , informational severity	Absence of an observer is a gap, not a bypass
Backpressure	Bounded per-stage queue; dropped units counted and reported per <code>request_id</code>	Lets the observer's coverage statement name what it missed
Payload limits	Same unit sizes; over-cap request bodies are delivered in units rather than failing the stage	An observer wants the bytes, not the decision
Encoding	Compressed bodies delivered as-is with <code>Content-Encoding</code> in the preflight, which the phase already carries	The observer can decompress; the relay should not have to
Registration	Advertised through <code>Describe</code> like any binding; attached by policy config per endpoint; runnable in a sandbox so its egress is policy-governed	No change to the policy model

What the observer derives itself

Nothing in section 4 needs a new field. `served_model` is in the response body. `wire_complete` is derivable from the request body (the presence of a stored-conversation pointer). Timestamps are the observer's own receipt times. The one field the runtime could add cheaply is a reference to the policy decision on the transaction, so a reading sits beside what was permitted; the OCSF stream already carries that decision keyed by the same `request_id` , so it may need no new field at all.

Findings and OCSF

The observer's findings use the contract's `Finding` as it stands: a stable type, a label without request content, a count, a confidence, a severity. One addition would make the output honest, and another middleware author has already asked for it in the project's tracker: a way for a stage to declare which dimensions it could not observe on a given invocation. The observer's OCSF events follow the runtime's rule: no body content, a SHA-256 hash for correlation, and `request_id` as the key. The observer's own record, which does hold copies, is its own store under the deployment's retention policy and is not an OCSF sink.

10. What the observer refuses to claim

The observer's credibility rests on the claims it will not make, and a runtime team evaluating it should hold it to the list.

- **It does not detect intent.** It measures displacement between two parties' accounts of an exchange and takes no position on why they diverged. A bad-faith party and a compressed summary produce the same kind of finding; the record is what lets someone else tell them apart.
- **It does not detect a lie that both parties believe.** A false premise planted by one party and sincerely absorbed by the other yields matching accounts. The instrument measures the propensity of model pairs to absorb planted premises in controlled runs; it does not detect a specific absorbed premise in production.
- **It does not read encoded coordination.** A referent that binds to nothing in the receiver's visible context is counted as unresolved, not decoded. Where the share of such referents rises between a pair, that is reported as a shape, not interpreted as a message.
- **It does not link paraphrase when linking exchanges.** Text is matched as it is or not at all, because a wrong link is a false fact on the record. Paraphrase is tolerated in comparison, where it is a reading task with a measured accuracy, not in linking, where it would be a guess.

- **It does not gate.** For actions whose harm cannot be repaired afterward, the right control is an approval gate, which is the runtime's instrument. The observer surfaces divergence while there is time to act and keeps the record by which the gates are themselves judged.

Each refusal is verifiable from the outputs. The coverage statement reports the unlinked, undeposed and unresolved fractions; the audit events carry a basis and confidence class on every claim; and the absence of any verdict field in the record is a property a reviewer can check rather than a promise.

11. What is outside this document

The observer's design specification, *Exchange Observation*, is available under a mutual non-disclosure agreement and is not described here. It covers the parts of the instrument that this interface statement names only by their outputs:

- how exchanges are linked without identifiers, and how each link's basis and confidence class are assigned;
- how the comparison is calibrated to a noise band per pair of model versions, and what a slip must exceed;
- how the reader that extracts commitments is qualified, and the protocols that measure its agreement with labelled examples;
- the propensity battery that measures accommodation, commitment assignment and hedge survival by model pair;
- the decisions taken against alternative designs, and the open questions.

Nothing in that specification changes the interface above. A runtime that exposes the hook contract in this document can carry the observer regardless of how the observer's internals evolve, and the coverage statement is the instrument's own account of what those internals achieved on the runtime's traffic.

Mossrake Group, LLC · mossrake.ai · The framework paper, *Meaning Slip: A Framework for AI Conversation Governance*, is public and is the authority for every claim about the instrument made in this document. © 2026 Mossrake Group, LLC. Mossrake®, Language Model Diligence™ and Language Model Tectonics™ are trademarks of Mossrake Group, LLC.